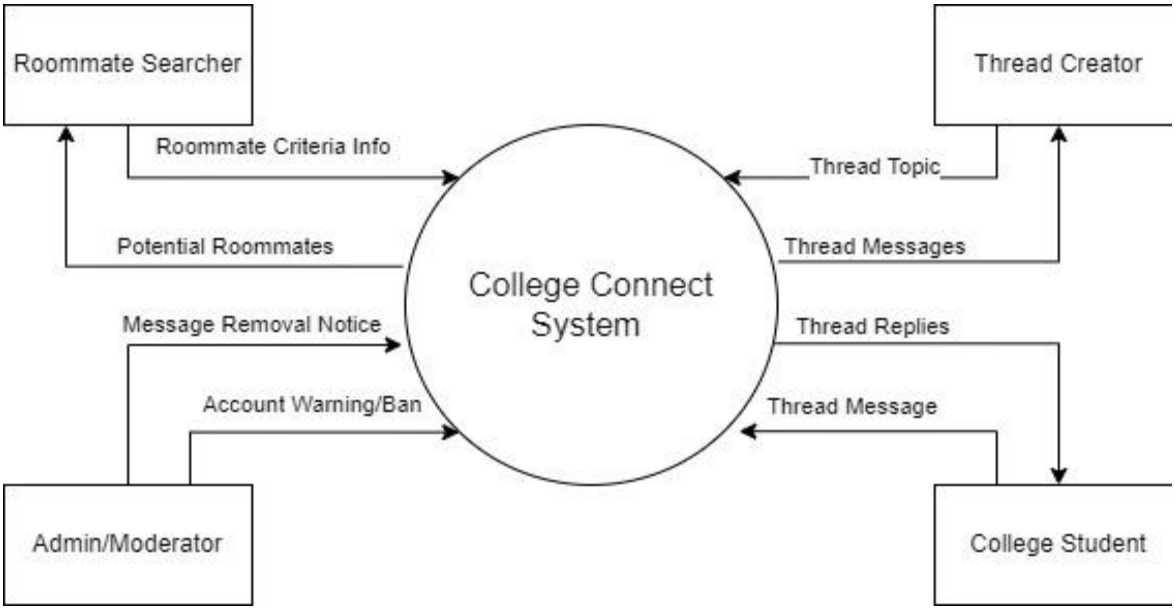
Project Overview

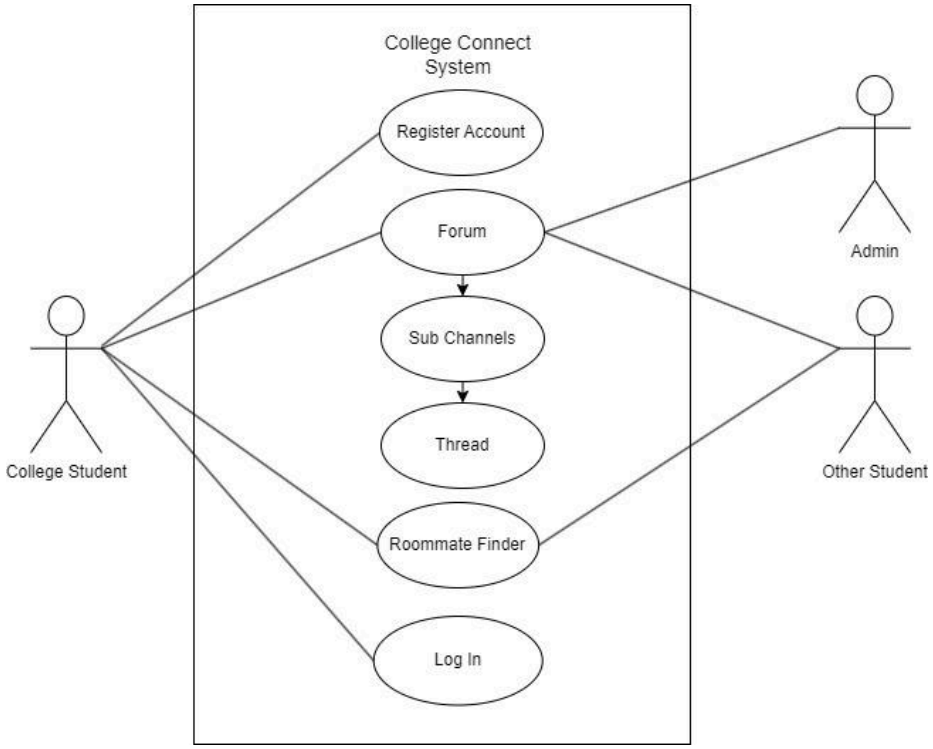
Our project is College Connect.
The purpose of this project is to provide University students with a safe and isolated platform to connect with their fellow peers. The project has two main features, Roommate Finder, a function that allows students to find optimal roommates and/or friends by inputting a set of preferred and deferred qualities in a peer, and a Forum that allows students to hold discussions freely on a message board that is subdivided by topic.
The problem our project wishes to address is the difficult part of the College experience which is connecting with a new set of peers. Roommates are a particularly difficult part of the equation. When roommates are decided at random, it may result in an uncomfortable living situation due to a clash of personalities and lifestyles. Thus, College Connect allows students to reach out to peers who would make optimal roommates to create an ideal living situation for everyone.
Our stakeholders for this project would be college faculty and staff who would decide to license the use of our software for the sake of increasing the quality of life for students.
User Stories:

- *I am a college student who is moving into dorms for a new semester, and I'm not comfortable with the idea of rooming with a stranger I don't know. I would like to have a way to find a potential roommate I would get along with before settling my housing situation.*
- *I'm a college student who's very religious and looking for a roommate who shares the same beliefs to maintain a peaceful environment. I'm looking for a roommate who doesn't have any drinking/smoking habits and keeps a clean and family-friendly environment for whenever my family visits.*
- *I am a student arriving on campus for the first time. I would like to connect with other students and communicate with them to make new friends or social connections. Being able to send messages on an open forum would be nice as I could meet many new people while connecting with those who have similar interests.*

Context Diagram:



Use-Case Diagram:



Product Backlog:

- **Account Creation and Login**
- Account Information and Tagging
    - Store information about a person's preferences and attributes in the database upon account creation.
- Profile Page "about me" Display and Editing.

    - Display preferences and traits about a user on the profile page based on what is present on that user in the database, and allow a user to edit their account database information from the Profile Page.
- Roommate Finder Search Function
    - Search for users whose preferences and traits match your criteria.
- User-to-User Direct Messaging
- Forum
    - Implement a Basic Forum Layout
    - Implement Thread Creation and the ability to post replies to a thread.
    - Add a feature to auto-sort threads by recent activity. (Time Permitting)
    - Forum Moderation features (Time Permitting)

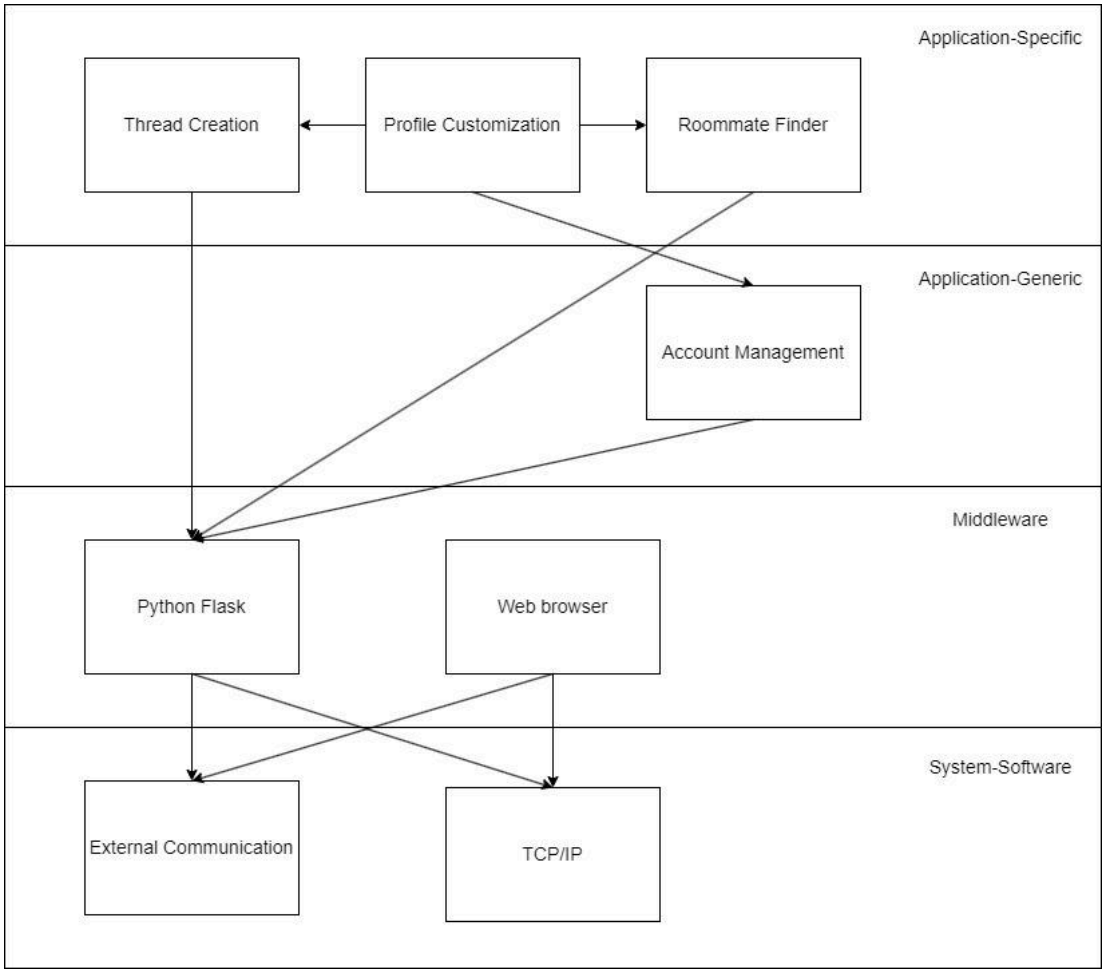
Main Use Cases:

- Roommate Finder
    - Assist College Students who are looking for the best possible roommates and peers to match their needs.
- Forum:
    - Allow students wishing to discuss their hobbies and interests or any concerns related to school a public platform to do so.
- User-to-User Direct Messaging
    - Gives a secure space for two users to have a private conversation with each other.

Architectural Overview

For our Architecture, we chose to do a simple client-server architecture model as that is the model that is most commonly used for websites. We didn't seriously consider any alternative designs due to a lack of familiarity with them and a poor understanding of how to properly implement them given our limited tools.

Subsystem Architecture - Dependency Diagram

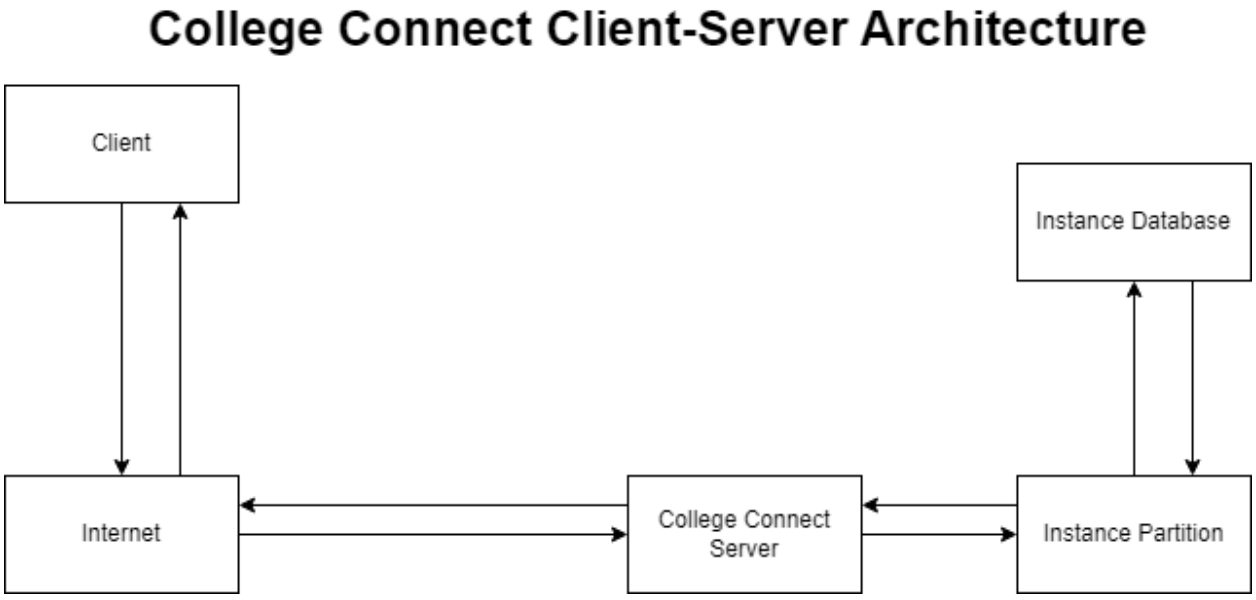


Thread creation is responsible for creating a new thread under a subcategory when a user requests it. Profile customization provides the means to customize information about the user and their profile picture. Roommate finder is responsible for searching for users that match the current user's preferences. Account management allows a user to manage or delete their account. The web browser is responsible for displaying the website and python flask provides the framework for both the database and frontend. As stated above we chose the client server style as it is what we are most familiar with.

Deployment Architecture

Our project would use client-server architecture.
When a college purchases a license to use our system, we set up a partition on our web server to host that college's own instance of the CollegeConnect website. This instance holds its own personal database that holds information relevant only to that instance. When a client logs on to the College Connect website, they will be prompted to enter their college email address and password, after which they will be redirected to their college's instance of CollegeConnect. (i.e CollegeConnect.UNCC.edu) Within this instance, the client can only communicate with other

users from within the same instance, with no opportunity to interact with users from a separate instance. All computation and handling of input and response should be handled by the College Connect servers, with the clients only viewing the responses on their computer monitors.



Persistent Data Storage

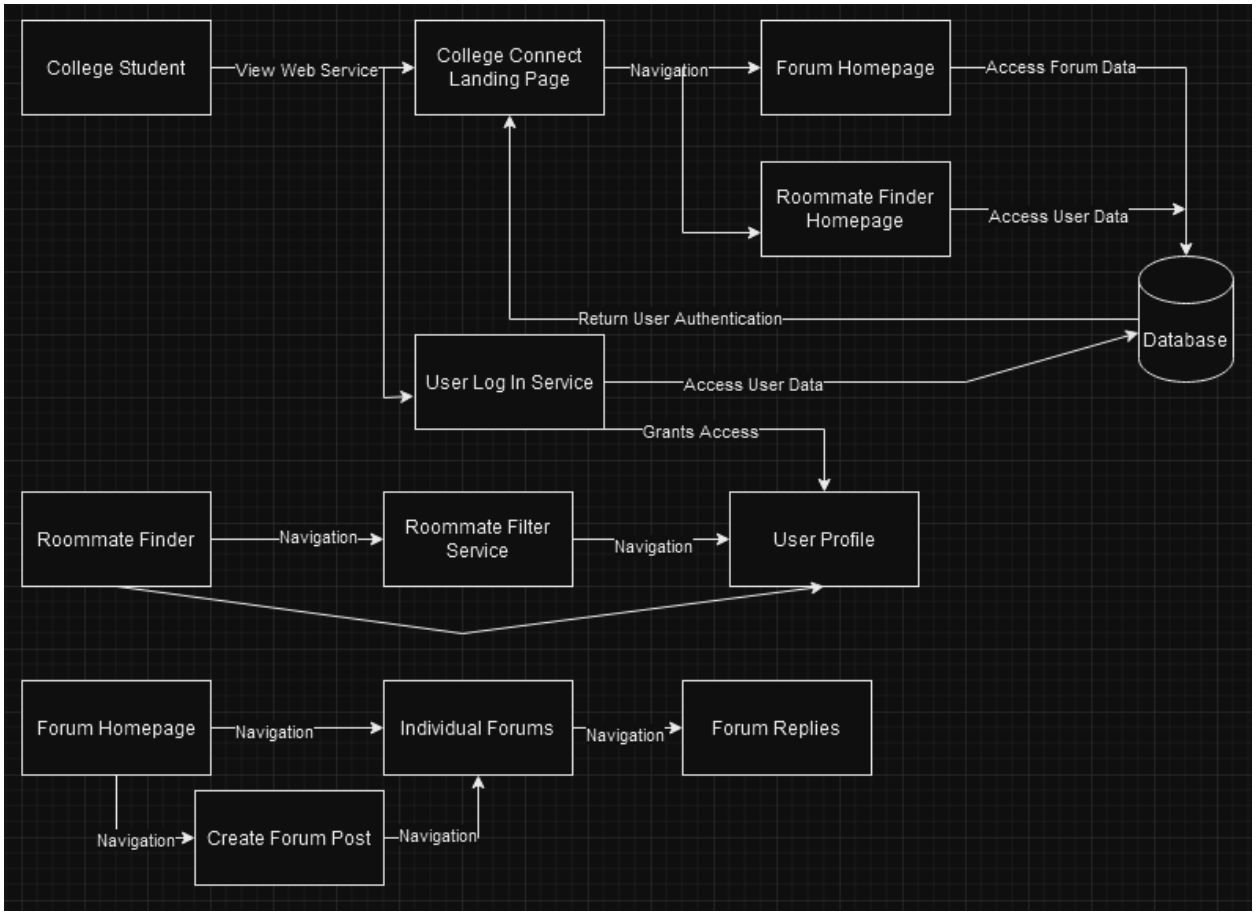
College Connect utilizes databases for persistent storage. Since we are using Python Flask for our application, SQLAlchemy is used to create a database and connect it to the College Connect website. The type of data stored will mostly be String and Integer values for personal information such as age, hobbies, college years, etc. Due to the fact that this website is meant for college students to find roommates, get help with college-related questions, or find people with similar interests, data related to personal preferences is what will be utilized. SQLAlchemy makes storing data rather simple, it will be stored in rows with data starting from the first column with account information such as username and followed up by personal attributes the user has chosen to share. This is how the database schema is represented within the code with only the login functionality currently implemented:

```
def __repr__(self):
    return f"Username: {self.username}, Password: {self.password}, Email: {self.email}"
```

Global control flow

The global control flow of our project will be mostly procedural as the process for forum thread creation and finding roommates will be rather linear. Event-driven control can be utilized in user comments on forum threads and admin features. There will be no timer-controlled actions in our system. Our system uses multiple threads since concurrent users are expected to access the database at the same time. Implementing a monitor to ensure that only one thread can execute at a time should help prevent damaging data in the database.

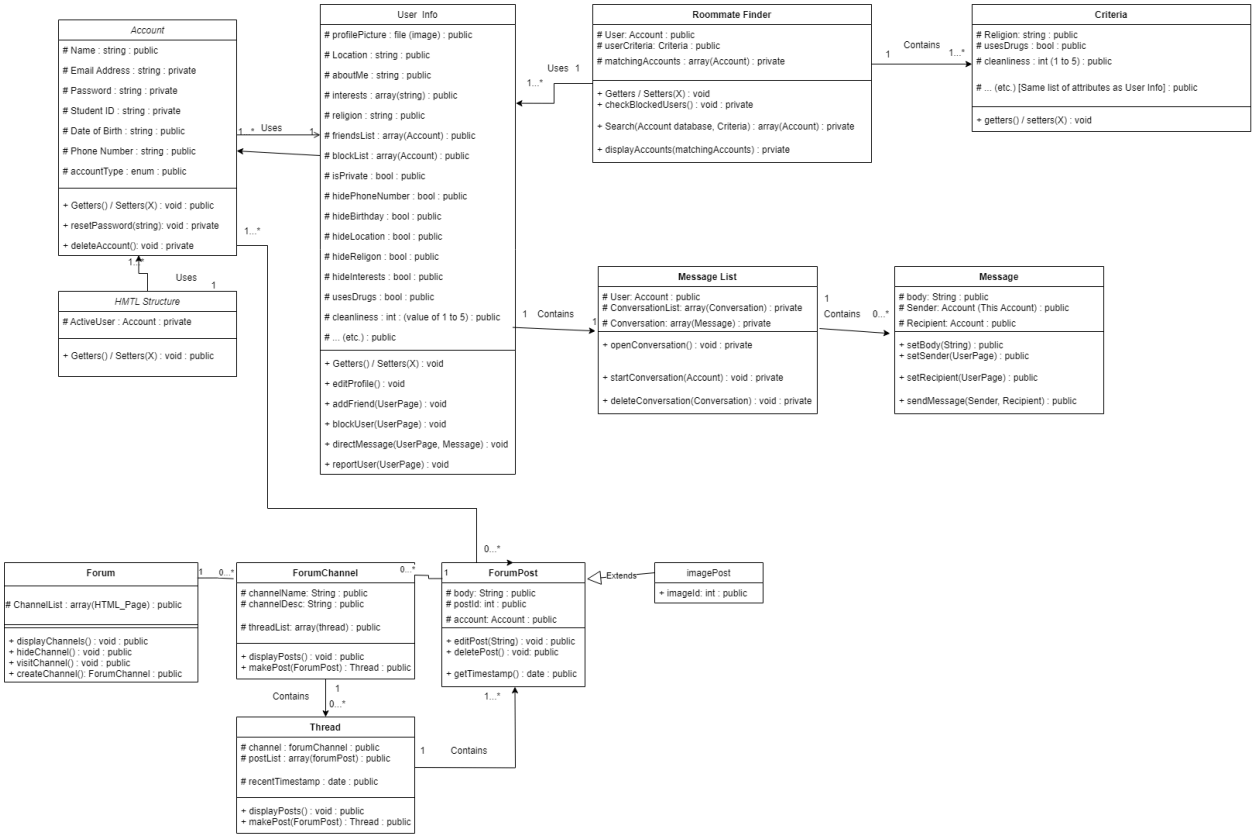
Detailed System Design



A simple high-level overview displaying the system architecture which outlines the project functionality. Architecture is further expanded upon in the following class diagram and UML diagrams. This system design incorporates a simple web server and database to store user information and posts that users make. This information is then accessed through the user navigating the system and interacting with the functionality provided to them. The architecture for the roommate finder feature as well as the forum system has been expanded upon as the design for those features is key to the project and must be understood between all team members in order to make the product successful.

Static view

Class Diagram (revised)



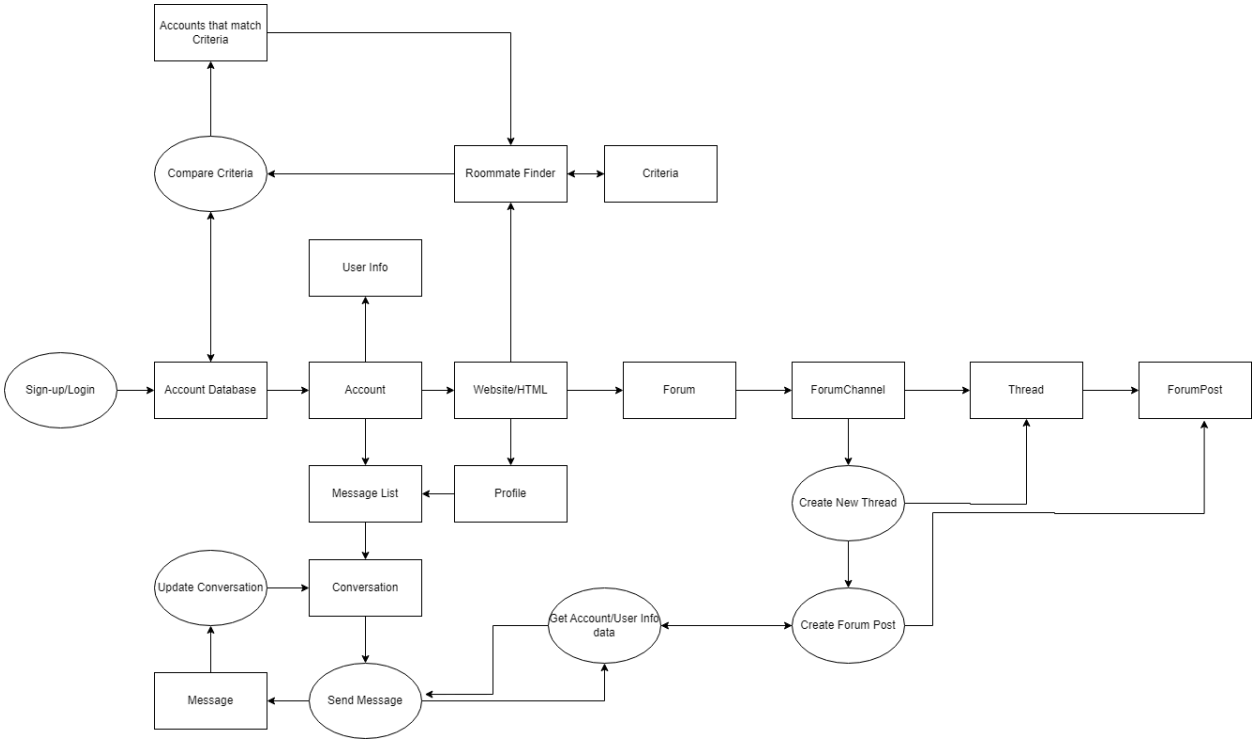
Our Class diagram is laid out in a way that centers around the user's account as most systems will have to call information from the user's account and user info in some way. The distinction between Account information and user info is that the information under user info is what is set by the editing feature and Account information is information set upon account creation and is not easily modifiable without contacting an administrator to update or change the info. The Account's name is what is most often needed for the purpose of attaching the user's identity to the HTML overlays, forum posts, and messages.

The Roommate Finder's responsibility is to read the criteria list created by the user compare in the User Info of other accounts and return accounts that match the criteria. The criteria object is effectively an isolated, simplified user info object with fields that can be compared to the User Info contained attached to an account.

The message list is another part of the Account's database that contains a list of all the different direct message conversations that a user has been having. Within the message list are the individual conversations, which are an array of message objects, which hold the text data sent from the sender account to the recipient account.

Forums are primarily responsible for containing a list of channels, and channels are responsible for containing a list of threads, which in turn are a list of forum posts. Forum posts contain the text and images a user wishes to send and identifying information about the user sending the forum post by carrying their account's name and profile picture.

Dynamic view



Our Current Sprint is dedicated to wrapping up the features related to Account Information in the database. Once the sign-up page asks the user for all the appropriate information necessary for the innate account fields, and once the user is able to view and edit their “user info” fields from their profile page, this sprint will be complete.
Each of our features is relatively independent once the account system is created.
Following this Sprint, we will begin to implement the Roommate Finder feature, as its only real dependency is on the account database having user info on file for each account, as well as experiment with a basic HTML layout for our Forum before developing it proper in the following Sprint.
In review of our previous Sprints, we believe that our project had scope issues, and needed to be scaled down and reworked. As a result, we’ve come out with a new perspective on trying to plan ahead in order to keep features as simplistic and minimal as possible to avoid feature creep from unexpected complexity. We are no longer trying to make an ideal program that matches our initial vision, but rather a minimalistic prototype that satisfies all our requirements.